

SDA - Seminar 6 - TD

Conținut:

- Iterator DO reprezentat sub formă de TD, cu rezolvare coliziuni prin liste independente
- Dicționar reprezentat sub formă de TD, cu rezolvare coliziuni prin liste întrepătrunse

1. Iterator pe Dicționar Ordonat (DO) reprezentat sub formă de tabelă de dispersie (TD), cu rezolvare coliziuni prin liste independente.

- Presupunem că:
 - Memorăm doar cheile
 - Avem chei numere naturale
 - Ordinea impusă cheilor este "<" (cheile fiind distincte)

Exemplu:

- Presupunem că dorim să adăugăm în dicționar următoarele 9 chei: 5, 28, 19, 15, 20, 33, 12, 17, 10
- , iar dicționarul este reprezentat pe o TD cu
 - $m = 9$ locații
 - și optăm pentru
 - dispersia prin diviziune



În cazul dispersiei prin diviziune, cum este definită funcția de dispersie d ($d(c)=?$) ?

✓ $d(c) = c \bmod m$



Care sunt valorile de dispersie pentru cele 9 chei care se doresc a fi adăugate în DO?

c	5	28	19	15	20	33	12	17	10
d(c)									

c	5	28	19	15	20	33	12	17	10
d(c)	5	1	1	6	2	6	3	8	1

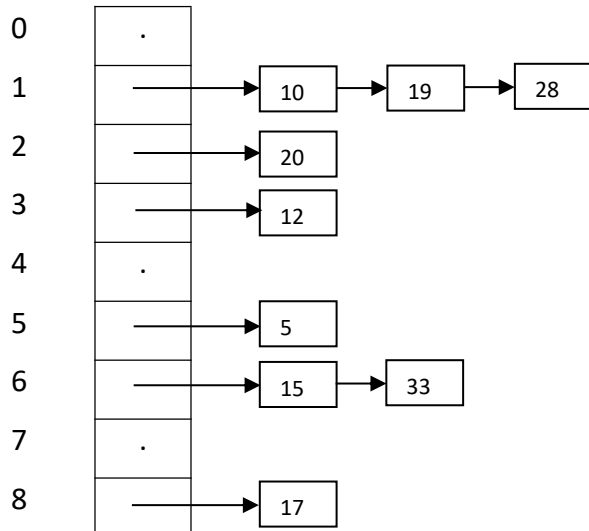
- **Observație:** Secvența d(c) formată de valorile de dispersie conține valori care se repetă. Acestea corespund coliziunilor. Având în vedere că discutăm în contextul unei TD cu rezolvare coliziuni prin liste independente, cheile aparținând unei coliziuni vor fi memorate în aceeași listă (independent de cheile având alte valori de dispersie).



Cum arată TD cu ajutorul căreia se reprezintă Dicționarul Ordonat, ulterior inserării cheilor exemplificate (5, 28, 19, 15, 20, 33, 12, 17, 10)?

0	()
1	()
2	()
3	()
4	()
5	()
6	()
7	()
8	()

TD:



Observație: Cheile se inserează în listele independente astfel încât să se respecte relația impusă Dicționarului Ordonat. Cu alte cuvinte, toate listele independente vor fi (independent) ordonate.



Care este ordinea de parcurgere a cheilor din DO care se așteaptă a fi oferită de Iterator?

- ✓ Elementele ar trebui să fie parcurse de Iterator în această ordine: 5, 10, 12, 15, 17, 19, 20, 28, 33, adică în ordine crescătoare.



Cum se obține această secvență ordonată a cheilor pornind de la TD ilustrată anterior?

- ✓ Se observă că lista ordonată a cheilor se obține prin interclasarea tuturor listelor independente (cu ajutorul cărora se memorează cheile care intră în coliziune).
Așadar, **implementarea Iteratorului pe DO se poate reduce la interclasarea prealabilă (în constructor) a listelor independente, într-o nouă listă, ordonată, urmată de parcurgerea ei.**



Considerând că reprezentăm listele independente ca Liste Simплу Înlanțuite (LSI) alocate dinamic, care este reprezentarea Dicționarului Ordonat?

Reprezentare DO:

NodT:

c: TCheie (*presupunând că memorăm doar cheile; altfel, adăugăm v: TValoare*)

urm: ↑NodT

DicționarOrdonat:

m: Intreg (*dimensiunea tabelii*)

l : (↑NodT)[] (*vectorul de liste independente*)

d: TFuncție: TCheie -> {0,1,...,m-1} (*funcția de dispersie*)

R: Relație: TCheie x TCheie -> {A, F} (*relația de ordine impusă cheilor*)



Cum putem reprezenta Iteratorul pe Dicționarul Ordonat?

IteratorDicționar:

d: DicționarOrdonat

l: ↑NodT (*pointer la primul nod al listei ordonate obținută prin interclasare*) (sau, alternativ, TListă)
curentNod: ↑NodT (*relația de ordine impusă cheilor*)



Cum implementăm constructorul iteratorului având această reprezentare?

```
subalgoritm creeaza(it, d):  
    it.d ← d  
    interclaseazaListe (d, it.l)  
    it.curentNod ← it.l (sau it.l.prim, depinzând de tipul ales,  
    al rezultatului operației interclasează)  
sf_subalgoritm
```



Care vor fi complexitățile timp ale operațiilor *valid, următor, element*?

✓ Operațiile *valid, următor, element* vor avea complexitate constantă: $\Theta(1)$.



Care sunt posibilitățile de interclasare ale listelor independente (de implementare a operației *interclaseazaListe*)?

✓ *interclaseazăListe* poate interclasa listele independente ordonate :

1. pe rând, adică interclasând prima listă cu a 2-a, după care lista rezultată din interclasarea primelor două liste cu cea de-a 3-a ș.a.m.d.
2. Concomitent, adică interclasând toate listele deodată folosind un ansamblu



Care va fi complexitatea interclasării (pentru fiecare dintre cele două variante menționate anterior)?

✓ Complexitatea interclasării:

$$\left. \begin{array}{l} \text{TD cu } m \text{ poziții} \\ \text{Dicționar cu } n \text{ elemente} \end{array} \right\} \Rightarrow \text{nr mediu de elemente într-o listă: } \frac{n}{m}$$

$= \alpha$ (factor de încărcare)

1. Interclasare prima lista cu a 2-a, etc.:

- lista1 + lista2 => lista12 ----> $\alpha + \alpha = 2\alpha$ (operații elementare)
- lista12 + lista3 => lista123 ----> $2\alpha + \alpha = 3\alpha$

- lista123 + lista4 => lista1234 ----> $3\alpha + \alpha = 4\alpha$
- ...

=> Se obțin, în total : $2\alpha + 3\alpha + \dots + m\alpha \approx \left. \frac{m(m+1)}{2} \alpha \right\}_{\alpha = \frac{n}{m}} \rightarrow \frac{m(m+1)}{2} \frac{n}{m} \Rightarrow \in \theta(n * m) \approx \theta(n)$

(presupunând m – constant)

2. Dicționar reprezentat pe TD, în varianta de rezolvare a coliziunilor prin liste întrepătrunse

- Presupunem, din nou, că:
 - Memorăm doar cheile (doar pentru ilustrarea exemplului)
 - Avem chei numere naturale

Exemplu:

- Presupunem că dorim să adăugăm în dicționar următoarele 7 chei: 5, 18, 16, 15, 13, 31, 26
- , iar dicționarul este reprezentat pe o TD cu
 - $m = 13$ locații
 - și optăm pentru
 - dispersia prin diviziune



Care sunt valorile de dispersie pentru cele 7 chei care se doresc a fi adăugate în Dicționar?

c	5	18	16	15	13	31	26
d(c)							

c	5	18	16	15	13	31	26
d(c)	5	5	3	2	0	5	0



Cum arată TD cu ajutorul căreia se reprezintă Dicționarul, ulterior adăugării cheii 5?

➤ Adăugăm cheia 5

- Valoarea de dispersie pentru 5 este 5
- Slotul având indexul 5 este liber
- Adăugăm cheia 5 în slotul având indexul 5
- `primLiber = 0` (rămâne neschimbat)

[illegible][illegible]



Cum arată TD cu ajutorul căreia se reprezintă Dicționarul, ulterior adăugării cheii 18?

➤ Adăugăm cheia 18

- Valoarea de dispersie pentru 18 este 5
- Slotul având indexul 5 este ocupat
- Prin urmare, adăugăm 18 pe slotul având indexul $\text{primLiber} = 0$
- $\text{primLiber} = 01$ (actualizăm valoarea primului slot liber)
- Având în vedere că 18 se dispersează pe indexul 5, începând de la indexul 5 trebuie să ajungem, urmând legăturile, la cheia 8 (deci la indexul 0). Cu alte cuvinte, adăugăm cheia 18 (de pe indexul 0) la finalul înlanțuirii începând de pe indexul 5 (egal cu valoarea ei de dispersie). Aceasta se rezumă la $\text{urm}[5]=0$.

	0	1	2	3	4	5	6	7	8	9	10	11	12
c	18					5							
urm	-1	-1	-1	-1	-1	0	-1	-1	-1	-1	-1	-1	-1



Cum arată TD cu ajutorul căreia se reprezintă Dicționarul, ulterior adăugării cheii 16?

➤ Adăugăm cheia 16

- Valoarea de dispersie pentru 16 este 3
- Slotul având indexul 3 este liber
- Adăugăm cheia 16 în slotul având indexul 3
- $\text{primLiber} = 01$ (rămâne neschimbat)

	0	1	2	3	4	5	6	7	8	9	10	11	12
c	18			16		5							
urm	-1	-1	-1	-1	-1	0	-1	-1	-1	-1	-1	-1	-1



Cum arată TD cu ajutorul căreia se reprezintă Dicționarul, ulterior adăugării cheii 15?

➤ Adăugăm cheia 15

- Valoarea de dispersie pentru 15 este 2
- Slotul având indexul 2 este liber

- Adăugăm cheia 15 în slotul având indexul 2
- $\text{primLiber} = 01$ (rămâne neschimbat)

	0	1	2	3	4	5	6	7	8	9	10	11	12
c	18		15	16		5							
urm	-1	-1	-1	-1	-1	0	-1	-1	-1	-1	-1	-1	-1



Cum arată TD cu ajutorul căreia se reprezintă Dicționarul, ulterior adăugării cheii 13?

➤ Adăugăm cheia 13

- Valoarea de dispersie pentru 13 este 0
- Slotul având indexul 0 este ocupat
- Prin urmare, adăugăm 13 pe slotul având indexul $\text{primLiber} = 1$
- $\text{primLiber} = 014$ (cel mai mic index mai mare decât 1, al unui slot liber)
- Având în vedere că 13 se dispersează pe indexul 0, începând de la indexul 0 trebuie să ajungem, urmând legăturile, la cheia 13 (deci la indexul 1). Cu alte cuvinte, adăugăm cheia 13 (de pe indexul 1) la finalul înlănțuirii începând de pe indexul 0 (egal cu valoarea ei de dispersie). Aceasta se rezumă la $\text{urm}[0]=1$.

	0	1	2	3	4	5	6	7	8	9	10	11	12
c	18	13	15	16		5							
urm	1	-1	-1	-1	-1	0	-1	-1	-1	-1	-1	-1	-1



Cum arată TD cu ajutorul căreia se reprezintă Dicționarul, ulterior adăugării cheii 31?

➤ Adăugăm cheia 31

- Valoarea de dispersie pentru 31 este 5
- Slotul având indexul 5 este ocupat
- Prin urmare, adăugăm 31 pe slotul având indexul $\text{primLiber} = 4$
- $\text{primLiber} = 0146$ (cel mai mic index mai mare decât 4, al unui slot liber)
- Având în vedere că 31 se dispersează pe indexul 5, începând de la indexul 5 trebuie să ajungem, urmând legăturile, la cheia 31 (deci la indexul 4). Cu alte cuvinte, adăugăm cheia 31 (de pe indexul 4) la finalul înlănțuirii începând de pe indexul 5 (egal cu valoarea ei de dispersie). Având în vedere că $\text{urm}[5]=0$, $\text{urm}[0] = 1$ și abia $\text{urm}[1]=-1$, aceasta se rezumă la $\text{urm}[1]=4$.

	0	1	2	3	4	5	6	7	8	9	10	11	12
c	18	13	15	16	31	5							

urm	-1	1	-1	4	-1	-1	-1	0	-1	-1	-1	-1	-1	-1
-----	----	---	----	---	----	----	----	---	----	----	----	----	----	----



Cum arată TD cu ajutorul căreia se reprezintă Dicționarul, ulterior adăugării cheii 26?

➤ Adăugăm cheia 26

- Valoarea de dispersie pentru 26 este 0
- Slotul având indexul 0 este ocupat
- Prin urmare, adăugăm 26 pe slotul având indexul $\text{primLiber} = 6$
- $\text{primLiber} = 01467$ (cel mai mic index mai mare decât 6, al unui slot liber)
- Având în vedere că 26 se dispersează pe indexul 0, începând de la indexul 0 trebuie să ajungem, urmând legăturile, la cheia 26 (deci la indexul 6). Cu alte cuvinte, adăugăm cheia 26 (de pe indexul 6) la finalul înlănțuirii începând de pe indexul 0 (egal cu valoarea ei de dispersie). Având în vedere că $\text{urm}[0]=1$, $\text{urm}[1] = 4$ și abia $\text{urm}[4]=-1$, aceasta se rezumă la $\text{urm}[4]=6$.

	0	1	2	3	4	5	6	7	8	9	10	11	12
c	18	13	15	16	31	5	26						
urm	-1	1	-1	4	-1	-1	0	-1	-1	-1	-1	-1	-1

Observații:

- primLiber se actualizează considerând pozițiile libere de la stânga la dreapta, ceea ce înseamnă că spațiul liber nu este înlănțuit, ci se gestionează secvențial
- Într-o înlănțuire putem avea elemente care aparțin unor coliziuni diferite. De exemplu, coliziunea care începe pe poziția 5: 5 (valoare dispersie: 5) → 18 (valoare dispersie: 5) → 13 (valoare dispersie: 0) → 31 (valoare dispersie: 5) → 26 (valoare dispersie: 0) conține chei care se dispersează atât pe poziția 5, cât și pe poziția 0.



Care este reprezentarea TAD Dicționar, considerând TD în varianta de rezolvare a coliziunilor prin liste întrepătrunse?

✓ **Reprezentare Dicționar:**

TElement:

c: TCheie

v: TValoare

Dicționar:

m: Întreg (dimensiunea tabelii)

e: (TElement)[]

urm:{0,..., m-1}[]

primLiber: Întreg {0,...,m}

d: TFuncție: TCheie → {0,..., m-1}

- **Observație:** În stabilirea domeniului valorilor pentru `primLiber`, s-a presupus că `primLiber=m` indică faptul că TD este plină.



Cum implementăm constructorul Dicționarului având această reprezentare?

```
subalgoritm creează (d):
    @ se inițiază funcție de dispersie d
    @ se inițiază m
    pentru i ← 0, m-1 execută
        d.e[i] ← NULL_TElement
        d.urm[i] ← -1
    sf_pentru
    d.primLiber ← 0
sf_subalgoritm
```



Care este complexitatea timp a operației *creează*?

✓ Complexitate: $\Theta(m)$, m reprezentând dimensiunea tabelii



Care este descrierea Pseudocod a operației de căutare din interfața Dicționarului?

```
Funcția caută(d, c):
    i ← d.d(c) //Calculăm valoarea de dispersie pentru cheia c
    cât timp (i ≠ -1 și d.e[i].c ≠ c) execută //Cât timp mai sunt
    elemente în înlanțuirea începând de la indexul egal cu valoarea de dispersie
    și nu am găsit cheia căutată
        i ← d.urm[i] //ne deplasăm la următorul element din înlanțuire,
    urmând legăturile
    sf_cât timp
    dacă i = -1 atunci //în cazul în care cheia nu a fost găsită
        caută ← NULL_TValoare //semnalăm aceasta returnând
    NULL_TValoare
    altfel
        caută ← d.e[i].v //altfel, returnăm valoarea asociată ei
sf_funcție
```



Care este complexitatea operației *caută*?

✓ Complexitate: $O(m)$, dar în medie (în ipoteza dispersiei uniforme simple) $\Theta(1)$

Observație: Implementarea operației de adăugare este descrisă în materialul aferent cursului 10, prin urmare continuăm cu operația de **ștergere**.

Exemplificăm operația de ștergere pentru cheia 5.

Pornim de la TD în această stare:

	0	1	2	3	4	5	6	7	8	9	10	11	12
c	18	13	15	16	31	5	26						
urm	1	4	-1	-1	6	0	-1	-1	-1	-1	-1	-1	-1

, indexul primului slot liber fiind `primLiber=7`.

Observație:

- Nu putem elibera slotul 5 marcând, totodată, `urm[5]` cu -1 deoarece am pierde legăturile înspre 18 și 31, nemaiputându-le găsi prin operația *caută*.

Ideea ștergerii (pentru exemplul considerat; descrierea generică este prezentată în materialul aferent cursului 10):

- Căutăm elemente, în înlănțuirea începând de la poziția de pe care ștergem (în cazul nostru, 5) care se dispersează pe această poziție (adică pe poziția 5).
 - a. Dacă nu există astfel de elemente, ștergem elementul precum am șterge un nod dintr-o listă simplu înlănțuită, refăcând legăturile
 - b. Dacă există un astfel de element, atunci mutăm elementul pe poziția de unde dorim să ștergem și repetăm procesul de ștergere pentru poziția de unde am mutat elementul.

Așadar:

- Dorim să ștergem cheia 5, care este pe poziția 5

	0	1	2	3	4	5	6	7	8	9	10	11	12
c	18	13	15	16	31	5	26						
urm	1	4	-1	-1	6	0	-1	-1	-1	-1	-1	-1	-1

- Căutăm primul element care se dispersează pe poziția 5. Din 5, urmând legătura (`urm[5]=0`), ajungem la cheia 18, care se dispersează întocmai pe poziția 5.
- Așadar, mutăm (copiem) 18 pe poziția 5

	0	1	2	3	4	5	6	7	8	9	10	11	12
c	18	13	15	16	31	5 18	26						

urm	1	4	-1	-1	6	0	-1	-1	-1	-1	-1	-1	-1
-----	---	---	----	----	---	---	----	----	----	----	----	----	----

- În continuare, dorim să ștergem cheia 18 de pe poziția 0 (de unde am identificat-o, aducând-o pe poziția 5)
- Căutăm primul element care se dispersează pe poziția 0. Din 0, urmând legătura (urm[0]=1), ajungem la cheia 13, care se dispersează întocmai pe poziția 0.
- Așadar, mutăm (copiem) 13 pe poziția 0

	0	1	2	3	4	5	6	7	8	9	10	11	12
c	18 13	13	15	16	31	5 18	26						
urm	1	4	-1	-1	6	0	-1	-1	-1	-1	-1	-1	-1

- În continuare, dorim să ștergem cheia 13 de pe poziția 1 (de unde am identificat-o, aducând-o pe poziția 0)
- Căutăm primul element care se dispersează pe poziția 1 (adică pe poziția de unde dorim să ștergem). Din 1, urmând legătura (urm[1]=4), ajungem la cheia 31 (de pe poziția 4), care nu se dispersează pe poziția 1. Din 4, urmând legătura (urm[4]=6), ajungem la cheia 26 (de pe poziția 6), care, din nou, nu se dispersează pe poziția 1. Cheia 26 este, însă, ultima cheie din înlanțuire, având în vedere că urm[6]=-1. Prin urmare, deducem că nu există o cheie care se dispersează pe poziția 1. Așadar, ștergem poziția 1 din înlanțuire (precum am șterge dintr-o LSI), modificând legăturile (urm[0]<--urm[1] = 4)

	0	1	2	3	4	5	6	7	8	9	10	11	12
c	18 13		15	16	31	5 18	26						
urm	1 4	4 -1	-1	-1	6	0	-1	-1	-1	-1	-1	-1	-1

Având în vedere că am eliberat un index mai mic decât primLiber=7, primLiber devine 1 (primLiber=~~7~~1).



Care este descrierea Pseudocod a operației *sterge*, conform ideii prezentate anterior?

subalgoritm *sterge*(d, c) **este**

```

i ← d.d(c) //calculăm valoarea de dispersie pentru cheia c
j ← -1 //rețin precedentul lui i, întrucât, când ștergem, ne este
necesar "nodul" anterior, pentru refacerea legăturii
//în următoarea structură repetitivă, parcurgem tabela pentru a verifica
dacă poziția i are vreun anterior
k ← 0

```

```

cât timp (k < d.m și j = -1) execută
    dacă d.urm[k] = i atunci
        j ← k
    altfel
        k ← k+1
sf_cât timp
    //în următoarea structură repetitivă, localizăm cheia care trebuie
    //ștearsă, actualizând și precedentul
    cât timp i ≠ -1 și d.e[i].c ≠ c execută
        j ← i
        i ← d.urm[i]
    sf_cât timp
    //verificăm existența cheii
    dacă i = -1 atunci
        @cheia nu există
    altfel //în caz afirmativ,
        //căutăm, în înlănțuirea începând de pe poziția i o primă altă cheie
        //care se dispersează în i (poziția de șters)
        gata ← fals //folosim o variabilă booleană care devine adevărat
        //când nu se dispersează nimic în i (poziția de șters)
        repetă
            p ← d.urm[i] //prima poziție verificată
            pp ← i //anteriorul poziției p
            cât timp p ≠ -1 și d.d(d.e[p].c) ≠ i execută //cât timp
            //mai avem chei în înlănțuire și nu am găsit o cheie care se dispersează pe
            //poziția i, avansăm în înlănțuire, urmând legăturile, cu poziția curentă și cu
            //cea anterioară ei
                pp ← p
                p ← d.urm[p]
            sf_cât timp
            dacă p = -1 atunci
                gata ← adevărat //marcăm situația în care nu există o
                //cheie care se dispersează în i (poziția de șters)
            altfel //altfel
                d.e[i] ← d.e[p] //aducem cheia (perechea) de pe poziția
                //p pe poziția i
                j ← pp //poziția anterioară noii poziții de șters devine
                //pp
                i ← p //iar noua poziție de șters devine p (poziția de pe
                //care am adus cheia pe fosta poziție de șters)
            sf_dacă
            până_când gata
            //nemaifiind chei care se dispersează pe poziția de șters, ștergem
            //această poziție din înlănțuire
            dacă j ≠ -1 atunci //în cazul în care are un precedent, refacem
            //legătura de la poziția precedentă (în înlănțuire) la următoarea poziție (în
            //înlănțuire)
                d.urm[j] ← d.urm[i]
            sf_dacă
            //și marcăm faptul că poziția i a fost eliberată

```

```
d.e[i] ← NULL_TElement
d.urm[i] ← -1
//în cazul în care am eliberat o poziție mai mică decât primLiber,
actualizăm valoarea primei poziții libere, dat fiind că gestionăm spațiul
liber de la stânga la dreapta.
    dacă d.primLiber > i atunci
        d.primLiber ← i
    sf_dacă
sf_dacă
sf_subalgoritm
```

De ce este necesară următoarea structură repetitivă?

```
k ← 0
cât timp (k < d.m și j = -1) execută
    dacă d.urm[k] = i atunci
        j ← k
    altfel
        k ← k+1
sf_cât timp
```

Rolul ei este de a identifica elementul precedent al poziției de dispersie i . Acest pas este necesar dacă o căutare în înlănțuirea care începe de la valoarea de dispersie nu ar găsi acest element precedent. Aceste situații pot apărea dacă efectuăm ștergeri succesive, care pot muta elemente pe poziția lor de dispersie, chiar dacă se află în interiorul unei înlănțuiri. Spre exemplu, dacă am vrea să ștergem elementul 13 după ce elementul 5 a fost șters: observăm că 13 se află pe poziția de dispersie (0), dar el are un element precedent (elementul 18). În exemplul nostru, 26 ar fi mutat în locul lui 13 și algoritmul ar fi corect, dar dacă elementul 26 nu ar exista și am șterge elementul 13 fără a ține cont de 18, o căutare ulterioară nu l-ar găsi pe 31. Observăm că această căutare secvențială este necesară doar dacă elementul nu are un element precedent în înlănțuirea care începe de pe poziția de dispersie (nu este obligatoriu să o efectuăm mereu). Algoritmul poate fi scris și astfel:

subalgoritm sterge(d , c) este

```
i ← d.d(c)
j ← -1
cât timp i ≠ -1 și d.e[i].c ≠ c execută
    j ← i
    i ← d.urm[i]
sf_cât timp

dacă i = -1 atunci
    @cheia nu există
altfel
    gata ← fals
    repetă
        p ← d.urm[i]
        pp ← i
        cât timp p ≠ -1 și d.d(d.e[p].c) ≠ i execută
            pp ← p
            p ← d.urm[p]
        sf_cât timp
        dacă p = -1 atunci
            gata ← adevărat
```

```

        altfel
            d.e[i] ← d.e[p]
            j ← pp
            i ← p
        sf_dacă
    până_când gata

dacă j = -1 atunci
    k ← 0
    cîttimp (k < d.m și j = -1) execută
        dacă d.urm[k] = i atunci
            j ← k
        altfel
            k ← k+1
    sf_cîttimp
sf_dacă

dacă j ≠ -1 atunci
    d.urm[j] ← d.urm[i]
sf_dacă

d.e[i] ← NULL_TElement
d.urm[i] ← -1
dacă d.primLiber > i atunci
    d.primLiber ← i
sf_dacă
sf_dacă
sf_subalgoritm

```

3) În seminarul 2 am discutat următoarea problemă: Se dă un număr k și un șir arbitrar de numere $x_1, \dots, x_n$. Determinați dacă există două numere în șir care au suma k .

Două dintre metodele discutate de rezolvare a acestei probleme se bazează pe ideea de a căuta, pentru fiecare i , valoarea $k - x_i$. Această observație ne poate duce cu gândul la a utiliza o tabelă de dispersie, pentru a obține o căutare mai eficientă. Astfel putem obține următoare metodă de a rezolva problema: folosim o tabelă de dispersie auxiliară. Pentru fiecare element x_i , verificăm dacă valoarea $k - x_i$ există în tabela de dispersie (a fost adăugată într-un pas anterior, adică există un $j < i$ cu proprietatea că $x_j = k - x_i$). Dacă răspunsul este afirmativ, înseamnă că am găsit o

soluție. În caz contrar, adăugăm valoarea x_i în tabela de dispersie. Dacă la finalul parcurgerii șirului nu am găsit valoarea $k - x_i$ pentru niciun i , înseamnă că nu există nicio pereche de elemente având suma k . Complexitatea timp în caz mediu pentru această rezolvare este $\theta(n)$.